

Findings: EIFBENCH: Extremely Complex Instruction Following Benchmark for Large Language Models

Tasneem Muneera Prottyasha

April 2026

Summary

The paper addresses the limitation of existing LLM evaluation benchmarks, which primarily focus on single-task or low-constraint scenarios and therefore fail to capture the complexity of real-world applications where models must handle multiple interdependent instructions under numerous constraints. To solve this, the authors propose EIFBENCH, a novel benchmark that incorporates multi-instruction, multi-constraint, and multi-type tasks, along with a new training method called Segment Policy Optimization (SegPO) that improves instruction-following by computing rewards at a finer, per-instruction level. The experimental process involves constructing a large-scale dataset (1,000 instances) through taxonomy design, data collection, task and constraint expansion, and rigorous quality control, followed by evaluation of 20 LLMs using metrics such as Instruction-Level Accuracy (ILA) and Constraint-Level Accuracy (CLA), with GPT-4o used as an automated judge. The results show that current LLMs perform poorly in fully satisfying complex instructions—especially at the instruction level—highlighting a significant gap in real-world capability, while SegPO improves performance by up to 14.85% over baseline models and outperforms GRPO. Overall, the analysis demonstrates that as instruction and constraint complexity increase, model performance degrades substantially, emphasizing the need for more advanced reasoning and optimization techniques to handle realistic multi-task workflows.

How does SegPo work?

SegPO is designed to improve how LLMs handle multiple instructions with multiple constraints—a setting where traditional methods (like GRPO) struggle because they treat the entire output as one unit. Instead, SegPO introduces fine-grained, instruction-level learning.

Main Idea

Instead of giving one overall reward for the entire response, SegPO:

- Breaks the output into segments, where each segment corresponds to a specific instruction
- Computes rewards separately for each segment
- Combines these with a global reward for better learning

Step-by-Step Mechanism

1. Segment the Output

The model generates a response for multiple instructions.
Each part of the response corresponds to a specific instruction.
These parts are treated as segments.

2. Compute Two Types of Rewards

(A) Global Reward (Overall Performance)

Measures how well the entire response satisfies:

- Instruction-Level Accuracy (ILA)
- Constraint-Level Accuracy (CLA)
- Output format correctness

This is similar to traditional RL reward.

(B) Segment Reward (Instruction-Level)

For each segment (instruction):

- Check if all constraints for that instruction are satisfied
- If yes $\rightarrow$ reward = 1
- If no $\rightarrow$ reward = 0

This is the key innovation: fine-grained feedback

3. Compute Advantages Separately

SegPO calculates two advantages:

- Global Advantage $\rightarrow$ based on overall reward
- Segment Advantage $\rightarrow$ based on instruction-specific reward

Then combines them:

$$A_{i,t} = A_{i,t}^{global} + A_{i,t}^{segment}$$

This means:

$\Rightarrow$ Each token learns from both global correctness and local instruction correctness

4. Policy Optimization

Uses RL (similar to GRPO)

Updates the model to:

- Improve overall response quality
- Improve each instruction's correctness individually

Why SegPO Works Better

Problem with GRPO

- Uses single reward for entire output
- If one instruction fails → weak feedback signal
- Model doesn't know which part went wrong

Advantage of SegPO

- Gives precise feedback per instruction
- Helps model:
 - Handle multi-task workflows
 - Satisfy multiple constraints simultaneously
 - Learn which specific part needs improvement

Result (from the paper)

- +14.85% improvement over base model
- +3.40% improvement over GRPO
- Much better handling of complex, real-world instructions

Intuition (Simple Analogy)

Think of grading a student:

- GRPO → Gives one final grade for the whole exam
- SegPO → Grades each question separately and gives a final score

SegPO helps the model learn exactly where it made mistakes

How They Actually Ran the Experiments

The prompts in the appendix are not used directly for evaluation; instead, they are used in dataset construction and the training pipeline.

1. Dataset Construction (Where prompts are heavily used)

They did not manually write all tasks. Instead, they used LLMs with structured prompts to generate the benchmark.

- **Task Expansion Prompt**

Input is a simple task or text. The prompt asks the LLM to generate multiple related tasks, include different task types, and introduce dependencies such as parallel, sequential, conditional, and nested. The result is multi-instruction tasks.

- **Constraint Expansion Prompt**

Takes each instruction and adds six to ten detailed constraints. These constraints are measurable and non-vague. The result is multi-constraint instructions.

- **Task Combination Prompt**

Combines the original input text and expanded tasks to produce one complex instruction containing multiple sub-instructions and constraints.

- **Validation Prompts**

Check feasibility, logical consistency, and redundancy, and remove invalid samples.

The outcome of this stage is the EIFBENCH dataset with one thousand instances. Each instance contains six to twelve instructions and more than seventy constraints on average.

2. Model Evaluation (Where prompts are used differently)

After constructing the dataset:

- Models are run on each complex instruction to generate outputs.
- An LLM is used as a judge to check whether each constraint is satisfied or not.
- Metrics are computed:
 - Constraint-Level Accuracy measures the percentage of constraints satisfied
 - Instruction-Level Accuracy measures whether all constraints of an instruction are satisfied

3. Training with SegPO (Where prompts are used in RL)

The training prompt instructs the model to solve each sub-instruction, follow constraints strictly, and use a structured format.

This structure enables:

- Identification of segments, where each sub-instruction is treated as a separate segment
- Computation of rewards by checking whether each segment satisfies its constraints

Reward Pipeline

- Global reward is based on instruction-level accuracy, constraint-level accuracy, and format correctness
- Segment reward is computed for each instruction: if all constraints are satisfied, the reward is one; otherwise, it is zero

Then advantages are computed and the SegPO update is applied.

Full Experimental Pipeline

- Generate dataset using prompts
- Run multiple LLMs on the dataset
- Evaluate outputs using an LLM judge
- Train the model using SegPO
- Compare performance across models

Key Insight

The prompts are used for dataset creation, data cleaning, training structure, and evaluation.

Subtle but Important Point

The pipeline is fully LLM-driven, where LLMs generate the benchmark, evaluate it, and are also used in training.

Why This Matters

This design is scalable and realistic, but it introduces potential evaluation bias because an LLM evaluates another LLM.

Role of Human Experts

- Humans are used for dataset quality control by checking contradictions, redundancy, infeasible tasks, and unclear constraints
- Humans also verify the reliability of the LLM judge by comparing human judgments with LLM outputs

Interpretation

The system is primarily LLM-driven, while human experts are used for validation and calibration rather than full evaluation.

Final Takeaway

LLMs generate and evaluate most of the data, while human experts act as a sanity check to ensure reliability.